

AI vs Human Text Detector

1. Project Definition

1.1 Problem Statement

Stakeholder's Problem: With the advancement of artificial intelligence in today's day and age, it is becoming more and more difficult to identify the difference between text written by humans and text generated by AI. AI models have the capability of writing essays, articles, emails, and more, which can closely resemble human writing. This raises concerns in areas such as education, journalism, or the online realm, where identifying the origin and authenticity of written content is important.

The goal of this project is to build a system using machine learning that can classify whether a piece of text was written by a human or generated by artificial intelligence. By analyzing characteristics of text such as sentence length, vocabulary diversity, punctuation usage, and repetition patterns, the system can learn patterns that distinguish human writing from AI-generated text. This project will store text data in a database, clean and analyze the dataset, create visualizations of writing patterns, and train a machine learning model that predicts whether text is AI-generated or human-written.

1.2 Connection to Course Material

This project connects directly to the topics covered in CS210 because it encompasses database management, data cleaning, data visualization, and machine learning. First, we use a relational database to store the text samples and extracted features so that the data can be organized, queried, filtered, and analyzed using SQL. Next, we used Pandas to clean the dataset by removing duplicates, handling missing values, fixing formatting issues, and preparing the text for analysis. After cleaning, we extracted data using algorithms to find word count, sentence length, punctuation usage, and vocabulary diversity. Moreover, these features helped us to analyze patterns, create visualizations using matplotlib, and train machine learning models like Logistic Regression, Decision Tree, and Random Forest to accurately classify text as human or AI-generated.

2. Novelty and Importance

2.1 Importance of the Project

There are a variety of problems in connection with the development of new technologies, especially AI. One of them is the possibility of producing human-like texts using advanced language models such as ChatGPT, GPT-4, Claude, etc. They allow generating very consistent and context-relevant text, which becomes hard to differentiate from human-written one. These concerns can affect many aspects of everyday life. For example, students' work can be affected by plagiarism and cheating, making it hard to tell what they actually know and how original their work is. In addition, in all sorts of media (journalism, social, etc), AI-generated materials can be used to create false narratives leading to the spread of misinformation. Finally the increase in the number of automated texts also raises the problem of spamming and fake news.

It shows that the goal of the project is to rebuild trust in human-written text by identifying what makes it unique/distinctive features. The analysis of certain elements of human-written texts, such as sentence patterns, vocabulary choice, etc., will enable distinguishing human-generated text from machine-produced ones. In addition to that, the research will help gain knowledge about artificial intelligence's possibilities in copying human language skills.

Prior related work?

A significant amount of previous work on differentiating between human and AI-written content has been performed. Recent papers have looked into the capabilities of humans and machine models for detecting AI-generated text, pointing out how increasingly hard the task becomes with improvements in natural language generation technologies.

2.3 Review of Related Work

The study titled “Do humans identify AI-generated text better than machines?” from ScienceDirect shows that humans have difficulty identifying machine-generated text. According to the results obtained in the study conducted among 63 lecturers, almost half of the machine-generated text was identified correctly. Moreover, there was little difference in performance between humans and automated detection models, and high-quality machine-generated text was particularly difficult to distinguish from human-generated one. The study concludes that people's perception of writing quality does not differ depending on whether the text was created by AI or a human.

In another study named “Detecting Artificial Intelligence–Generated Versus Human-Written Medical Student Essays: Semirandomized Controlled Study,” available through PubMed Central, it was established that participants accurately identified artificial intelligence-generated essays in 70 percent of cases. The findings indicate that stylistic characteristics such as redundancy, repetition, and coherence are more influential than factual errors. In other words, linguistic properties should be considered when differentiating between machine-generated and human texts.

It is also worth mentioning the research by the Penn State Information Knowledge and Web Lab conducted by Dongwon Lee. The authors argue that the gap between human-written and machine-generated text narrows fast day by day. In summary, the available studies indicate that neither human assessment nor detection algorithms are flawless in terms of detecting AI-generated content. While considerable advances have been made in this domain, there remains no fail-proof method for identifying AI-written material.

3. Progress and Contribution

3.1 Data Utilization

In this study, we used a dataset that consists of human-written and AI-generated texts retrieved from publicly available sources, such as Kaggle. The initial dataset was in unstructured raw text form, but we converted it into a structured format to allow for better analysis and modeling.

In terms of managing our data, we saved the dataset in a relational database, where each record consisted of a text sample and a label (0 or 1), indicating whether it is human-generated (0) or AI-generated (1). Then, we upgraded the dataset by adding extra features such as word count, sentence count, average sentence length, vocabulary diversity, and punctuation frequency, stopword ratio, and average word length. Transforming this text into numerical features helped us to extract meaningful patterns from the data and prepare it for modeling (ML).

Next we had to ensure the data was of quality therefore needed to preprocess or in other words clean it. First, we deleted any duplicated records to avoid potential biases and got rid of any rows with missing values (NaN) to prevent mistakes, inaccuracy, or errors during the machine learning process. Finally we normalized the text data by making it lowercase and deleting excess spaces in order to maintain a consistent format so data can be analyzed more accurately. This was a key step in making sure the dataset was consistent and accurate for when we test and train out data.

Moving on to the Data Extraction part, we extracted features/data from the table to be used for analysis. We developed methods to calculate features such as word count, sentence count, and average sentence length to capture

structure. We also decided to measure vocabulary diversity, which shows how varied and/or unique the word usage, and punctuation frequency, which exemplifies how often punctuation marks appear in the text. Also we included features like average word length, stopword ratio, and paragraph count to capture more subtle and specific patterns in writing for deeper analysis. In all, turning such text into measurable data allowed us to compare human vs ai generated writing.

In addition we implemented a custom command-line interface text classification tool. This tool allows users to input their own text, which is then processed through the same way as our csv file incorporating feature extraction pipeline. The extracted features are then scaled in order to make sure all feature values are in the same standardized range as the trained data. Next this data is passed into the ML model to generate a prediction. The system outputs whether the text is classified as human-written or AI-generated, along with a dataframe of extracted feature values, the scaled feature values after normalization, the raw prediction output from the model, and the probability scores for each class. This component demonstrates how the model can be used in a real-world setting.

Data/Feature Extraction:

```
# Extracts numerical features from each text entry
def extract_data(text):
    words = text.split() # .split() method splits the text into a list of words based on whitespace
    word_count = len(words) # returns the number of items in a list (in this case, the number of words)
    unique_words = len(set(words)) # set is a data structure that only keeps unique values

    # Split text into sentences using punctuation marks like ., !, ?, etc.
    sentences = re.split(r'[.!?]+', text)
    # remove empty sentences
    sentences = [s for s in sentences if s.strip()]

    # num of sentences is the amount of sentences in text
    sentence_count = len(sentences)

    # average words per sentence
    # total num of words divided by total num of sentences
    # ==> EXAMPLE: 200 words / 10 sentences = 20 words per sentence
    avg_sentence_length = word_count / max(sentence_count, 1)

    # Percentage of unique words used in text
    # take unique words and divide by total word count to find a PROPORTION
    # FOR EX: if theres 100 words and 50 are unique proportion is 50/100 = 0.5 which means 50% of the words are unique --> multiply by 100 to get percentage
    vocab_diversity = (unique_words / max(word_count, 1)) * 100

    # Finds punctuation characters such as ., !, ?, commas, quotes, etc.
    total_pun_chars = re.findall(r'[^\w\s]', text)

    # percentage of punctuation characters in text
    # take the number of punctuations there are and divide by total number of characters in text to PROPORTION
    # FOR EX: if theres 100 characters and 10 are punctuation proportion is 10/100 = 0.1 which means 10% of the characters are punctuation --> multiply by 100 to get percentage
    # max() prevents division by zero by ensuring the denominator is at least 1 NOT REALLY NECESSARY but just a safety measure
    punctuation_freq = (len(total_pun_chars) / max(len(text), 1)) * 100

    # average number of characters per word
    # EXAMPLE: "utilize" (7) vs "use" (3) -> AI avg_word_length will be higher
    avg_word_length = sum(len(w) for w in words) / max(word_count, 1)

    # stopwords are common filler words that do not add much meaning to a sentence, such as "the", "a" "is" etc.
    # percentage of words that are stopwords (filler words like "the", "and", "I")
    stopwords = {'the', 'a', 'an', 'is', 'in', 'it', 'of', 'and', 'to', 'was', 'that', 'for', 'on', 'are', 'with', 'as', 'at', 'be', 'this', 'by', 'from'}
    stopword_ratio = (sum(1 for w in words if w in stopwords) / max(word_count, 1)) * 100

    # number of paragraphs in the text
    # paragraphs are separated by one or more blank lines (\n\n)
    paragraphs = [p for p in re.split(r'\n\s*\n', text) if p.strip()]
    paragraph_count = max(len(paragraphs), 1) # at least 1 paragraph

    return {
        'word_count': word_count,
        'sentence_count': sentence_count,
        'avg_sentence_length': avg_sentence_length,
        'vocab_diversity': vocab_diversity,
        'punctuation_freq': punctuation_freq,
        'avg_word_length': avg_word_length,
        'stopword_ratio': stopword_ratio,
        'paragraph_count': paragraph_count,
    }
```

Numerical data cleaning example:

```
def clean_df(df):
    df = df.drop_duplicates() # removes all rows that are copies of eachother
    df = df.dropna(subset=['text']) #removes any rows where "text" col is blank
    df = df.dropna(subset=['generated']) #removes any rows where any "generated" col is blank

    df['text'] = df['text'].str.strip().str.lower() # convert text to lowercase and remove any whitespace
    return df

def setup_database():
```

3.2 Models, Techniques, and Algorithms

We used data visualization techniques to gain insights and help visualize our data. Using Matplotlib, we first created bar charts to compare extracted features between human and AI-generated texts which allows us to identify patterns and relationships between said features and its label (human or ai). Thus, these visualizations helped us determine which features were most useful for classification models.

Based on these insights, we applied 3 machine learning algorithms which varied in complexity which would then allow us to compare their performance and determine which approach worked best. These machine learning algorithms consisted of Logistic Regression, Decision Tree, and Random Forest because they matched our goal of comparing models with different levels of complexity and interpretability.

Afterwards, the dataset was then split into training and testing sets to evaluate model performance on unseen data. We decided to experiment with multiple train-test splits such as 70/30, 80/20, and 90/10, to observe how each model performed under different conditions. Next we applied 5-fold cross-validation which is a technique that splits data into multiple parts and tests models multiple times to ensure the results were not dependent on a specific subset of the data. Additionally we performed hyperparameter tuning on the Decision Tree and Random Forest model which involves testing different tree depths and number of trees to find a combination which model performs best at.

After training the models, we generated additional bar charts to visualize all 3 models' performance and feature importance for the Random Forest model because it provides a built-in and reliable method for measuring feature importance across multiple trees. Overall these visualizations helped us to evaluate how well each model performed and understand which features contributed most to the final predictions.

3.3 Experimental Design

We predicted that AI-generated text would show higher levels of consistency, structure, and briefness compared to human-written writing which tends to be more varied and less uniform. For example, human writing was expected to have higher word count, sentence count, paragraph count, and punctuation frequency due to detailed, sometimes redundant, and less structured or concise nature. On the other hand AI-generated text was expected to have higher vocabulary diversity, longer average word length, and slightly higher stopword usage due to its more formulaic, polished, and concise style. Additionally, we expected more complex models like the Random Forest to perform better since it combines multiple decision trees allowing it to capture more detailed patterns.

Based on these predictions our end goal was to test whether these extracted features could effectively distinguish between human-written and AI-generated text. In order to do so, we used machine learning models to analyze how well these extracted features captured patterns and differences within the writing style and if it could be used to accurately classify the text.

3.4 Key Findings and Results

Based on the analysis results, there were evident differences between texts written by humans and those generated by AI. Among the models considered, Random Forest had the highest performance. Its accuracy score ranged from 98 to 99 percent. Similarly, Decision Tree had relatively high accuracy scores ranging from 96 to 97 percent. On the contrary, Logistic Regression had low accuracy scores ranging from 84 to 85 percent. This exemplifies that the performance of models with higher complexity performed better than the simpler ones. Moreover the accuracy scores remained consistent among the models across the different train-test splits (70/30, 80/20, and 90/10).

To further validate our results we used 5-fold cross-validation. On average the Logistic Regression had an accuracy score of 0.8478 (84.78%), the Decision Tree had 0.9659 (96.59%), and Random Forest had 0.9871 (98.71%). Additionally, another significant finding we came across was how the most important feature to distinguish between AI and human text was the **average word length** followed by word count, punctuation frequency, vocabulary

diversity, average sentence length, paragraph count, stopwords ratio, and sentence count. In conclusion these findings support our predictions and suggest that linguistic patterns play an important role in classifying texts generated by humans and AI.

3.5 Evaluation

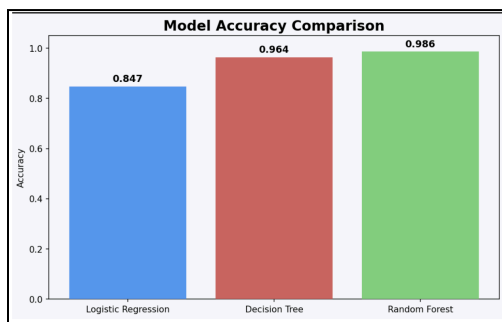
The performance of the models was tested based on metrics such as accuracy, precision, recall, and F1-score. According to the outcomes, the best-performing model was Random Forest, because it demonstrated high levels of precision and recall for each class. It means that this classifier is capable of distinguishing human-written and artificial texts with minimum mistakes.

Decision Trees were seen to demonstrate decent performance however they were less consistent and less accurate than Random Forest. In addition, Logistic Regression failed to recall some of the patterns for AI-generated text. Overall, the results of our analysis show that models that combine multiple individual models (ensemble classifiers) end up performing better because they are able to capture more complex features and nonlinear relationships in comparison to single models.

3.6 Advantages and Limitations

One of the major benefits of this approach is that it uses interpretable extracted features which makes it possible for us to understand how the model differentiates between texts created by a person and those created by AI systems. Moreover, applying several models and various validation methods (cross val and hyperparameter tuning) helped to ensure the results are reliable.

However, there are certain limitations that should be mentioned. Firstly, AI-created texts are becoming more and more similar to texts created by people. Secondly, the data collection strategy might not be representative. Moreover, the approach pays attention only to the superficial features of texts without considering their deeper meaning.



PART 4: FEATURE IMPORTANCE		
Feature Importance		
5	avg_word_length	0.409182
0	word_count	0.117296
4	punctuation_freq	0.104585
3	vocab_diversity	0.081520
2	avg_sentence_length	0.077933
7	paragraph_count	0.074517
6	stopword_ratio	0.068048
1	sentence_count	0.066920

4. Changes After Proposal

4.1 Differences from Proposal

In our first proposal, we had intended to gather data using web scraping and APIs. But because of time restrictions and the lack of access to dependable sources, we opted to use a dataset that exists online on Kaggle. This shift in strategy enabled us to spend more time processing/cleaning the data, training model, and creating custom CLI features, rather than having to allocate much of our time collecting data. Although our approach made data collection relatively easy, it gave us a pretty realistic set of data for detecting discrepancies between human-written and computer-generated texts.

4.2 Bottlenecks and Challenges

One of the biggest challenges we had was dealing with the problem of unstructured text data. Transforming the data into useful numeric features needed proper planning and testing. The second challenge we had was the problem of

overfitting, which could occur due to lack of enough training samples or complex models. To address this concern of ours we split our dataset into several train-test splits and employ cross-validation techniques. There were other technical difficulties in setting up the project on different platforms, especially with regards to file paths and dependencies especially with Mac systems and windows. These issues required modifying the code to ensure compatibility across systems.

Moreover we also had difficulties implementing the custom command line interface text classification part. One issue was that the model performed poorly on short or informal user inputs since they were different then the longer text the data was trained on leading to inconsistent predictions. Next we struggled to make sure the scaling process used during training was consistently applied to the user input. Although the CLI tool is still functional the model we used relies more heavily on engineered numerical features and was trained on more structured text data. However, the purpose of the CLI is to represent a real-world application of the model and demonstrate how the system can be used interactively, rather than to serve as a perfectly accurate tool.

5. Conclusion and Future Work

5.1 Summary of Contributions

For this project, we created an end-to-end process for detecting whether text was written by humans or generated by artificial intelligence. The first step involved preprocessing raw data and saving it into a structured database. Then, we converted unstructured text into useful numeric features. Next, based on these features, we analyzed the dataset via visualization and trained various machine learning algorithms, such as Logistic Regression, Decision Tree, and Random Forest. Using different train-test partitions, cross-validation techniques as well as hyperparameter tuning we compared the performance of each algorithm to determine which one is best suited for the task. This project illustrates the process of converting raw text data into useful insights to building an accurate classification model. Overall, we found that it was possible to distinguish between the two classes of text based on linguistic features.

5.2 Future Directions

Future developments for this project include employing larger datasets to enhance generalization. In addition we can also make improvements by applying more complex natural language processing algorithms, like Transformer-Based Classifiers or N-gram Analysis which have the capability to identify deeper semantic structures. Moreover another effective technique would be to analyze 30+ linguistic features yet for sake of model running time we stuck to smaller numbers. Finally, the project can also be used to develop a real-time application where users can input text and generate predictions regarding whether it was generated by humans or artificial intelligence.